

A certified outer bound for convex NLP scenario subproblems

mpi-sppy

companion to doc/designs/ipopt_outer_bound_design.md

Abstract

The usual device for getting outer bounds in mpi-sppy is to read the dual bound off the subproblem solver, which fails for Ipopt, which reports no dual bound. This note derives the bound that the `ipopt_outer_bound` spoke computes instead. The construction is Lagrangian weak duality combined with a tangent-plane underestimator minimised in closed form over the variable box. *None of the mathematics is new*: Theorem 3 is the Frank–Wolfe duality gap on a box, and Section 8 places each result against the literature. The derivation is written out because the hypotheses are what matter in practice, and one of them is easy to get backwards. Its defining property is that it requires *no* assumption about the accuracy of the solve: a truncated or sloppy solve yields a loose bound rather than an invalid one, and an ill-conditioned one does the same, since the construction evaluates rather than solves and so admits no amplification of error by a condition number. Convexity, by contrast, is genuinely load-bearing, and Section 5 isolates the one place where it is easy to get backwards.

1 Setting

We assume minimization. Fix a scenario s with probability p_s , $\sum_s p_s = 1$. Let $v \in \mathbb{R}^n$ collect *all* variables of the scenario subproblem — the nonanticipative variables $x(v)$ together with the recourse variables — and let

$$B = \prod_{i=1}^n [l_i, u_i]$$

be the box of variable bounds. With hub weights W_s , the subproblem an outer bound spoke solves is

$$L_s(W_s) = \min_v \{ f_s(v) + W_s^\top x(v) : g_s(v) \leq 0, h_s(v) = 0, v \in B \}, \quad (1)$$

with $f_s : \mathbb{R}^n \rightarrow \mathbb{R}$, $g_s : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $h_s : \mathbb{R}^n \rightarrow \mathbb{R}^k$. This is exactly the problem the ordinary Lagrangian spoke builds, with the proximal term switched off.

Why the returned objective value is not a bound here. For a minimisation, the value the solver reports is the objective evaluated at the point it stopped at. Any such value is $\geq$ the minimum, so it is an *inner* bound: the wrong direction. Nor does Ipopt’s convergence tolerance close the gap, since it is a relative KKT-residual tolerance rather than a bound on the objective error, and the two error sources point opposite ways — optimality error stops slightly above the optimum, while constraint violation can place the iterate slightly below it.

Remark 1 (A convex QP where Ipopt reports `optimal` and its value is not a bound). The objection to the preceding paragraph is that the subproblems are convex, so a solve reporting `optimal` has found the minimum. Consider the Hilbert QP with no variable bounds,

$$\min \frac{1}{2} x^\top H x - \mathbf{1}^\top x \quad \text{s.t.} \quad \mathbf{1}^\top x \leq 5, \quad H_{ij} = 1/(i+j+1),$$

which is convex, since $H \succ 0$. With the bounds dropped the single constraint is active and no other active set has to be determined, so the KKT conditions form one linear system and the optimum is available in exact rational arithmetic. Ipopt reported `optimal` in every row below.

n	exact optimum	$\ x^*\ _\infty$	$\ \hat{v}\ _\infty$	Ipopt's value	value – optimum
10	$-39/8$	3.50e5	3.50e5	-4.874999577980	$+4.2e-7$
12	$-1415/288$	8.66e6	8.70e6	-4.912905741134	$+2.9e-4$
14	$-1935/392$	2.09e8	1.82e5	-4.912217323377	$+2.4e-2$
16	$-2535/512$	5.56e9	1.93e5	-4.924764205425	$+2.6e-2$

Every entry in the last column is positive: each reported value lies above the optimum, so none of them is an outer bound. Two mechanisms are at work. At $n = 10$ the optimizer is not at fault — $\hat{v}$ is within a rounding of x^* , and $f(\hat{v})$ in exact arithmetic is -4.875000044826 , *below* the optimum, because $\hat{v}$ carries a little constraint violation. The $+4.2e-7$ enters when f is evaluated in double precision at $\|x\| \sim 3.5e5$, where individual terms reach $4.35e9$ and cancel down to 4.875 ; this is Remark 11, not a convergence failure, and tightening `tol` does not reach it. At $n = 14$ and $n = 16$ the solve stops three to four orders of magnitude short of x^* and reports success anyway, the termination test being applied to the scaled KKT residual; there the overshoot is an optimality error, and $2.6e-2$ on a quantity of size 5 is wrong in the third significant digit. Both arrive labelled `optimal`, and nothing else in the solver's output distinguishes them from a well-behaved solve. On a well-scaled model with modest variable magnitudes the returned value usually is close enough to pass unnoticed; the claim is not that it is always wrong, but that nothing reported says when.

2 Weak duality

Lemma 2 (Weak duality). *Let $\lambda \in \mathbb{R}_+^m$ and $\mu \in \mathbb{R}^k$ be arbitrary. Define*

$$\varphi_s(v) = f_s(v) + W_s^\top x(v) + \lambda^\top g_s(v) + \mu^\top h_s(v), \quad (2)$$

$$q_s(\lambda, \mu) = \inf_{v \in B} \varphi_s(v). \quad (3)$$

Then $q_s(\lambda, \mu) \leq L_s(W_s)$.

Proof. Let v be any point feasible for (1). Then $v \in B$, so $q_s(\lambda, \mu) \leq \varphi_s(v)$. Moreover $g_s(v) \leq 0$ and $\lambda \geq 0$ give $\lambda^\top g_s(v) \leq 0$, and $h_s(v) = 0$ gives $\mu^\top h_s(v) = 0$; hence $\varphi_s(v) \leq f_s(v) + W_s^\top x(v)$. Chaining the two inequalities, $q_s(\lambda, \mu) \leq f_s(v) + W_s^\top x(v)$ for every feasible v . Taking the infimum over feasible v gives the claim. $\square$

Note what Lemma 2 does *not* require: λ and μ need not be optimal, or even good. Any $\lambda \geq 0$ and any μ will do. This is the property that makes the whole approach robust, and it is why a mistaken sign convention or a stale multiplier can only cost tightness (Remark 5).

The trap. Lemma 2 is not yet usable, because q_s is defined by an *infimum*. Handing φ_s to a nonlinear solver returns a *point*, and the value there is $\geq$ the infimum — an upper bound on q_s , which is the wrong direction again. A second NLP solve therefore does not by itself certify anything.

3 The box underestimator

Theorem 3 (Certified bound). *Let U be an open convex set containing both B and $\hat{v}$, and suppose φ_s is convex on U and differentiable at $\hat{v}$. Define*

$$\hat{q}_s(\hat{v}) = \varphi_s(\hat{v}) + \sum_{i=1}^n \inf_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i). \quad (4)$$

Then $\hat{q}_s(\hat{v}) \leq q_s(\lambda, \mu) \leq L_s(W_s)$, and each term of the sum is available in closed form:

$$\inf_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i) = \begin{cases} \partial_i \varphi_s(\hat{v}) (l_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) > 0, \\ \partial_i \varphi_s(\hat{v}) (u_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) < 0, \\ 0, & \partial_i \varphi_s(\hat{v}) = 0. \end{cases} \quad (5)$$

The right-hand side is read in the extended reals: a term is $-\infty$ exactly when $l_i = -\infty$ with $\partial_i \varphi_s(\hat{v}) > 0$, or $u_i = +\infty$ with $\partial_i \varphi_s(\hat{v}) < 0$. The bound is then vacuous rather than invalid; Section 7 says what the implementation does with it.

Proof. Write $c_i = \partial_i \varphi_s(\hat{v})$ and let m_i denote the i -th infimum in (4), so that $\hat{q}_s(\hat{v}) = \varphi_s(\hat{v}) + \sum_i m_i$. There are three steps: a tangent-plane inequality on B , a coordinatewise lower bound on its linear term, and the evaluation of each m_i .

Step 1: the tangent plane lies below φ_s on B . Fix $v \in B$. Both v and $\hat{v}$ lie in the convex set U , so for $\theta \in (0, 1]$ convexity gives $\varphi_s(\hat{v} + \theta(v - \hat{v})) \leq (1 - \theta) \varphi_s(\hat{v}) + \theta \varphi_s(v)$, which rearranges to

$$\varphi_s(v) - \varphi_s(\hat{v}) \geq \frac{\varphi_s(\hat{v} + \theta(v - \hat{v})) - \varphi_s(\hat{v})}{\theta}.$$

Differentiability at $\hat{v}$ makes the right-hand side converge to $\nabla \varphi_s(\hat{v})^\top (v - \hat{v})$ as $\theta \downarrow 0$, so

$$\varphi_s(v) \geq \varphi_s(\hat{v}) + \sum_{i=1}^n c_i (v_i - \hat{v}_i) \quad \text{for every } v \in B. \quad (6)$$

Step 2: bound the linear term coordinate by coordinate. Because B is a box, $v \in B$ means $v_i \in [l_i, u_i]$ for each i , so $c_i(v_i - \hat{v}_i) \geq m_i$ by the definition of an infimum. No m_i is $+\infty$, so $\sum_i m_i$ is well defined in the extended reals, and summing over i in (6) gives $\varphi_s(v) \geq \hat{q}_s(\hat{v})$ for every $v \in B$. Taking the infimum over $v \in B$ yields $q_s(\lambda, \mu) \geq \hat{q}_s(\hat{v})$, and Lemma 2 supplies $q_s(\lambda, \mu) \leq L_s(W_s)$.

Step 3: the closed form. The function $t \mapsto c_i(t - \hat{v}_i)$ is affine. If $c_i > 0$ it is increasing, so its infimum over $[l_i, u_i]$ is its value $c_i(l_i - \hat{v}_i)$ at $t = l_i$ when l_i is finite, and $-\infty$ when $l_i = -\infty$. If $c_i < 0$ it is decreasing, and the same holds with u_i in place of l_i . If $c_i = 0$ it is identically zero. This is (5), including the cases in which a term is $-\infty$. $\square$

Corollary 4. $\hat{q}_s(\hat{v}) \leq L_s(W_s)$ for any $\hat{v} \in U$ at which φ_s is differentiable, any $\lambda \geq 0$ and any μ . In particular $\hat{v}$ need not be optimal, and need not even be feasible for (1).

Corollary 4 is the whole point. The certificate consumes the iterate the solver happened to stop at and the multipliers it happened to report, and returns a valid bound regardless. Computationally it costs one gradient evaluation and a loop over the variables — no second solve. It is worth reading the corollary as covering *ill-conditioned* solves and not merely truncated ones; Remark 10 says why the distinction does not arise here.

Remark 5 (Robustness). Because Corollary 4 holds for arbitrary $\lambda \geq 0$ and μ , an error in the sign convention used to recover the multipliers, a stale multiplier, or a multiplier clipped to zero can only make $\hat{q}_s$ smaller. Such an error makes the bound loose, never invalid. Convexity enjoys no such protection; see Section 5.

4 Exactness at a KKT point

The correction term in (4) is not merely small in practice: it vanishes identically at an exact KKT point, so the certificate degrades gracefully and costs nothing when the solve is good.

Proposition 6 (The correction vanishes). *Let $\hat{v}$ satisfy the KKT conditions of (1) with multipliers $\lambda \geq 0$ for $g_s \leq 0$, μ for $h_s = 0$, and $z_L, z_U \geq 0$ for the bounds $l - v \leq 0$ and $v - u \leq 0$. Then every term of the sum in (4) is zero, and if in addition φ_s is convex then $\hat{q}_s(\hat{v}) = L_s(W_s)$.*

Proof. Stationarity of the full Lagrangian, which is φ_s augmented with the bound terms, reads

$$\nabla \varphi_s(\hat{v}) = z_L - z_U. \quad (7)$$

Fix a coordinate i and consider the three possible positions of $\hat{v}_i$.

If $l_i < \hat{v}_i < u_i$, complementarity gives $z_{L,i} = z_{U,i} = 0$, so $\partial_i \varphi_s(\hat{v}) = 0$ by (7) and the term is 0 by (5).

If $\hat{v}_i = l_i$, complementarity gives $z_{U,i} = 0$, so $\partial_i \varphi_s(\hat{v}) = z_{L,i} \geq 0$. When the derivative is strictly positive (5) selects $t = l_i = \hat{v}_i$, and when it is zero the term is zero outright; either way the term is 0.

If $\hat{v}_i = u_i$, symmetrically $z_{L,i} = 0$ and $\partial_i \varphi_s(\hat{v}) = -z_{U,i} \leq 0$, so (5) selects $t = u_i = \hat{v}_i$ and the term is 0.

Hence $\hat{q}_s(\hat{v}) = \varphi_s(\hat{v})$. Complementarity $\lambda^\top g_s(\hat{v}) = 0$ and feasibility $h_s(\hat{v}) = 0$ reduce (2) to $\varphi_s(\hat{v}) = f_s(\hat{v}) + W_s^\top x(\hat{v})$. Under convexity the KKT conditions are sufficient for global optimality of (1), so that value is $L_s(W_s)$. $\square$

Proposition 6 has a practical corollary worth stating plainly: the correction term *measures its own inexactness*. It is zero when the solve is exact and grows as the solve degrades, so the reported bound loosens smoothly rather than becoming wrong.

5 Canonical form, and where convexity is easy to get backwards

Theorem 3 requires φ_s convex, and by (2) that requires each component of g_s to be convex and each component of h_s to be affine, since $\lambda \geq 0$ but μ is free in sign.

The subtlety is that g_s is the constraint in *canonical* form $g_s(v) \leq 0$, and putting a $\geq$ row into canonical form negates its body. Writing $b(v)$ for the constraint body as the user stated it:

as written	canonical form	requirement on b
$b(v) \leq \beta$	$g = b - \beta$	b convex
$b(v) \geq \alpha$	$g = \alpha - b$	b concave
$\alpha \leq b(v) \leq \beta$	both rows	b affine
$b(v) = \beta$	$h = b - \beta$	b affine

So $x^2 \leq 4$ is admissible and $x^2 \geq 1$ is not, though both are written with a convex body — and indeed the feasible set of the second is not convex. This is the one place in the construction where an entirely ordinary-looking model silently leaves the hypotheses, and the consequence is not a loose bound but an invalid one. A worked instance: for

$$\min x \quad \text{s.t.} \quad x^2 \geq 1, \quad x \in [0, 1.5],$$

whose optimum is 1, taking $\hat{v} = 0.5$ and $\lambda = 1$ in (4) yields $\hat{q} = 1.25 > 1$.

Of the four rows above, *two* are decidable — the two that demand an affine body, since affineness is a question about polynomial degree — and the implementation enforces those two. The other two rows are the one-sided ones, and each asks a question that is not decidable here: whether the body of a $\leq$ row is convex, and whether the body of a $\geq$ row is concave. Both remain user assertions, and the second is the trap above — a $\geq$ row with a convex body passes every mechanical check and breaks the theorem.

6 Aggregating across scenarios

The per-scenario bounds are combined by the usual probability-weighted sum, and the condition under which that sum is itself a bound deserves care.

Proposition 7 (Aggregation). *Let OPT be the optimal value of the original stochastic program. If the weights satisfy $\sum_s p_s W_s = 0$, then $\sum_s p_s L_s(W_s) \leq \text{OPT}$.*

Proof. Let x^* together with the scenario recourse decisions be optimal for the original problem, so that $\text{OPT} = \sum_s p_s f_s(v_s^*)$ where v_s^* is the optimal decision in scenario s and $x(v_s^*) = x^*$ for every s by nonanticipativity. Each v_s^* is feasible for subproblem (1), so $L_s(W_s) \leq f_s(v_s^*) + W_s^\top x^*$. Multiplying by p_s and summing,

$$\sum_s p_s L_s(W_s) \leq \sum_s p_s f_s(v_s^*) + \left(\sum_s p_s W_s \right)^\top x^* = \text{OPT} + 0. \quad \square$$

Remark 8 (Weights must share a generation). The hypothesis $\sum_s p_s W_s = 0$ is a *joint* condition on the whole weight vector, and progressive hedging maintains it at each iteration. It is not preserved by mixing: if W'_s takes scenario s 's weights from one iteration and scenario t 's from another, then in general $\sum_s p_s W'_s \neq 0$, the proof above leaves the residual term $(\sum_s p_s W'_s)^\top x^*$ in place, and the resulting number is not a bound at all. Nothing downstream can detect this, which is why a scenario whose certificate is unavailable must make the whole expectation unavailable rather than contribute a stale value.

7 Practical consequences

Tightness is governed by the box. By (5) the shortfall of $\hat{q}_s$ below $\varphi_s(\hat{v})$ is

$$\sum_{i=1}^n |\partial_i \varphi_s(\hat{v})| \cdot (\text{distance from } \hat{v}_i \text{ to the far end of } [l_i, u_i]),$$

so the bound is only as tight as the variable bounds are. Gradient components at a converged solve are small but generically nonzero, so a box of width 10 costs little while a box of width

10^{10} can cost a substantial fraction of the objective. Tightening bounds before certifying — for instance by feasibility-based bounds tightening, which shrinks B without removing feasible points and therefore only raises $\hat{q}_s$ — is the most effective way to improve the bound.

Remark 9 (Admissible tightenings of the box). The box in (5) need not be the one the model was written with, and the implementation shrinks it by two different mechanisms. The condition under which this is legitimate is not that a smaller box removes points — that argument shows the bound gets tighter, which is the direction that could break it — but the following. Let x^* be an optimal solution of the full problem. If $x^* \in B_s$ for every s , then for each s

$$\hat{q}_s \leq \inf_{v \in B_s} \varphi_s(v) \leq \varphi_s(x^*) \leq f_s(x^*) + W_s^\top x^*,$$

the last step because $\lambda \geq 0$, $g_s(x^*) \leq 0$ and $h_s(x^*) = 0$, and Proposition 7 goes through unchanged. The requirement is therefore that *every B_s contain one common optimal solution of the full problem*; preserving a different optimiser in each scenario does not suffice, since the weighted sum telescopes only at a single x^* .

Two sufficient conditions are used. Feasibility-based bounds tightening satisfies the requirement in the strong form: it removes no point satisfying the constraints of subproblem s , so B_s retains every feasible point of (1) and in particular x^* , with no appeal to optimality. Optimality-based tightening — reduced-cost fixing, which does discard feasible points — satisfies only the weak form, and relies on its own contract that some optimal solution survives, together with the fact that the same bounds are applied to every scenario, so that the surviving solution is common to all of them. A tightening that guarantees neither is not admissible, and no check in the implementation can detect that.

Unbounded variables. If $l_i = -\infty$ and $\partial_i \varphi_s(\hat{v}) > 0$, or $u_i = +\infty$ and $\partial_i \varphi_s(\hat{v}) < 0$, the corresponding term in (5) is $-\infty$ and no finite bound is available. Whether this occurs depends on the model and on how converged the solve is: at an exact KKT point the offending component is zero by Proposition 6, but away from one it generally is not. The implementation reports “no bound” in that case rather than $-\infty$, so that Remark 8 is respected.

Inexact solves. Corollary 4 already covers these, but it is worth spelling out what “inexact” can mean. Problem (1) is convex by hypothesis, so it has no non-global local minima. A solver returning a sub-optimal answer can therefore only mean that it stopped short of converging, leaving an inexact $\hat{v}$ and inexact multipliers. Both are admissible inputs to Theorem 3: the conclusion $\hat{q}_s \leq L_s(W_s)$ is unchanged, and the entire effect appears in the correction term of (4), which grows as $\hat{v}$ moves away from a KKT point. By Proposition 6 that term vanishes exactly at a KKT point, so the correction is precisely the price of inexactness, and it measures itself.

Remark 10 (Conditioning). The distinct worry is that (1) is badly conditioned, so that the solve is not merely truncated but numerically poor. This affects the tightness of $\hat{q}_s$ and not its validity, for a structural reason: *the certificate is an evaluation, not a solve*. A condition number quantifies how much a linear solve amplifies error — $\kappa(H)$ bounds the relative error in d obtained from $Hd = -g$ — and no step of (5) inverts anything. The computation evaluates φ_s at $\hat{v}$, evaluates $\nabla \varphi_s(\hat{v})$, compares each component with zero, selects an endpoint of the corresponding interval, multiplies and adds. Each is a forward evaluation, and κ has no route

in. Correspondingly, the inequality the construction rests on, the tangent-plane inequality (6),

$$\varphi_s(v) \geq \varphi_s(\hat{v}) + \nabla\varphi_s(\hat{v})^\top(v - \hat{v}) \quad \text{for every } v \in B,$$

is a pointwise consequence of convexity on U . It holds exactly, at every $\hat{v} \in U$ at which φ_s is differentiable, with no error constant and no dependence on the conditioning of anything. Conditioning governs how loose the plane is away from $\hat{v}$; it cannot change which side of φ_s the plane lies on. What it does change is the size of $\nabla\varphi_s(\hat{v})$ and the quality of the multipliers, and both push $\hat{q}_s$ down.

Floating point. Nothing in Theorem 3 requires a tolerance. The arithmetic, being carried out in finite precision, does introduce error, and the previous paragraph says only that this error is not amplified by κ — not that it is absent.

Remark 11 (Rounding). Let u denote the unit roundoff. Evaluating (4) in floating point incurs

$$|\text{fl}(\hat{q}_s) - \hat{q}_s| \lesssim u \left(|f_s(\hat{v})| + \sum_i \lambda_i |g_i(\hat{v})| + \sum_j |\mu_j| |h_j(\hat{v})| + \sum_i T_i d_i \right),$$

where T_i bounds the intermediate magnitudes arising in $\partial_i\varphi_s(\hat{v})$ and d_i is the distance from $\hat{v}_i$ to the far end of $[l_i, u_i]$. No condition number appears.

Remark 11 is governed by the size of the *summands*, so cancellation matters: multipliers of order 10^8 against an objective of order one put the error floor near 10^{-8} even though $|\hat{q}_s|$ is of order one. The implementation’s cushion, reporting $\hat{q}_s - \varepsilon(1 + |\hat{q}_s|)$, is scaled to the size of the *result* instead, and the two agree only on a well-scaled model. The cushion is therefore a numerical safeguard and not part of the argument, and it is exposed as a user-settable quantity rather than fixed.

Note also that the final term of Remark 11 is the one that can raise $\hat{q}_s$. The correction in (5) equals $-\sum_i |\partial_i\varphi_s(\hat{v})| d_i$, so a gradient component computed slightly small in magnitude makes it slightly less negative. Remark 5 gives no protection here: it is a statement about the multipliers, and a perturbation of $\nabla\varphi_s$ is not one-directional in the way a perturbation of λ is.

Non-finite values. A solve that diverges can leave NaN in $\hat{v}$ or in the reported multipliers, and an infinite multiplier yields NaN through $\infty \cdot 0$ or an infinite $\varphi_s(\hat{v})$. These propagate silently. The implementation therefore rejects any non-finite $\hat{q}_s$ and reports “no bound”, the same disposition it takes for an unbounded direction, so that Remark 8 is again respected. The case that makes this necessary rather than cosmetic is $+\infty$: a consumer keeping the best outer bound seen would discard NaN by accident, since NaN fails every comparison, but would accept $+\infty$ as an improvement.

8 Relation to known results

Nothing in Sections 2–7 is new. This section attributes each result.

Lemma 2 is textbook Lagrangian weak duality, stated for an arbitrary multiplier pair rather than an optimal one.

Theorem 3 is the *Frank–Wolfe duality gap*, sometimes called the Wolfe gap or the linearisation gap. For a convex f on a compact convex D , evaluating the linear minimisation oracle at x gives

$$f(x) + \min_{y \in D} \nabla f(x)^\top (y - x) \leq \min_D f,$$

which is (4) with $D = B$. The bound is used in the Frank–Wolfe algorithm [1], where the same oracle evaluation that produces it also supplies the search direction; reading it instead as a *certificate* computable at any iterate is the framing in [2]. The only specialisation here is that a box makes the oracle separable and closed-form, so the certificate costs one gradient evaluation and no optimisation at all.

Proposition 7 is the standard argument by which progressive hedging weights yield a lower bound on the optimal value, as in [3].

The remarks of Section 7 are elementary rather than novel. Remark 11 is the textbook forward-error bound for a floating-point sum, specialised to the summands of (4), and Remark 9 is two applications of Lemma 2. They are stated because the hypotheses they turn on are easy to lose sight of in implementation, not because they are results.

The combination of progressive hedging with Frank–Wolfe machinery to obtain Lagrangian dual bounds is itself established [5], and mpi-sppy already ships an FWPH cylinder on that basis.

What is specific here is therefore an assembly rather than a result: the multipliers are the ones Ipopt already returns, the linearisation point is the iterate it already stopped at, and the box is the model’s own variable bounds — so a valid outer bound is obtained from the solve the spoke was going to perform anyway, with no second optimisation and no assumption that the first one converged. The wider question of extracting valid dual bounds from an inexact oracle is treated at length in the inexact bundle-method literature, which this note does not attempt to survey.

9 Correspondence with the implementation

object here	in the code
φ_s in (2)	<code>_lagrangian_expression</code>
$\hat{q}_s$ in (4)	<code>certified_lower_bound</code>
decidable hypotheses	<code>check_model_is_certifiable</code>
box tightening, strong form	<code>unbounded_variables (fbbt)</code>
Remark 11 cushion	<code>eps_rel</code>
non-finite rejection	<code>math.isfinite</code> test on $\hat{q}_s$
box tightening, weak form	<code>receive_nonant_bounds</code>
Proposition 7 sum	<code>SPOpt.Ebound</code>
Remark 8 safeguard	a <code>None</code> bound, propagated collectively

Everything above the rule is in `utils/dual_certificate.py`, whose `eps_rel` argument is exposed on the command line as `--ipopt-outer-bound-cushion`. Below it, the weak-form tight-

ening is in `cylinders/spcommunicator.py` and the sum is in `spopt.py`, with the `None` propagated across both. All paths are relative to `mpisppy/`, and the spoke driving the whole is `cylinders/ipopt_outer_bound.py`.

A Why the reduced costs are not a shortcut

Ipopt does not report reduced costs. It reports the two vectors of bound multipliers $z_L \geq 0$ and $z_U \geq 0$ separately, and in this appendix the reduced cost is defined to be $z_L - z_U$, the combination that (7) equates with the gradient at a KKT point. (How Pyomo combines them is different; see Remark 13.) A natural proposal is to read the distance below the returned objective value off the multipliers directly, saving the gradient evaluation that Theorem 3 calls for. This appendix records why the implementation does not do that. The short version is that the reduced costs would enter the certificate in the place of the gradient, and equal it only at a KKT point; away from one, using the reported multipliers $z_L - z_U$ in place of the gradient $\nabla\varphi_s(\hat{v})$ evaluated at the returned point loses validity in a way that is systematic rather than occasional.

A.1 The reduced cost is the gradient

Equation (7) in the proof of Proposition 6 is exactly the identification: at a KKT point, $\nabla\varphi_s(\hat{v}) = z_L - z_U$, so the reduced cost of variable i is $\partial_i\varphi_s(\hat{v})$, and the distance below $\varphi_s(\hat{v})$ that must be conceded is the correction term of (4). Specialising φ_s to a linear function turns (4) into the textbook reduced-cost dual bound of linear programming,

$$c^\top \hat{x} + \sum_j \min\{d_j(l_j - \hat{x}_j), d_j(u_j - \hat{x}_j)\},$$

with d the vector of reduced costs; (4) is that bound's nonlinear generalisation. So the proposal is not a cheaper substitute for Theorem 3. It is Theorem 3, written in multiplier variables instead of gradient variables. In particular there is no second solve to be avoided: Corollary 4 already costs one gradient evaluation and a loop over the variables.

What a reduced-cost implementation would genuinely save is that gradient evaluation, replaced by a read of the solver's reported bound multipliers. That is a real cost — it is an automatic-differentiation sweep over φ_s — but it is the cost of a gradient, not of a solve.

A.2 Why the substitution is not valid

Equation (7) is a property of an exact KKT point, which is a hypothesis of Proposition 6 and not a fact about an arbitrary iterate. At a truncated solve the two sides differ by the dual-infeasibility residual

$$r = \nabla\varphi_s(\hat{v}) - (z_L - z_U) \neq 0. \tag{8}$$

Theorem 3 rests on the gradient inequality, which holds for a true (sub)gradient of a convex function and for nothing else. Replacing $\nabla\varphi_s(\hat{v})$ by $z_L - z_U$ therefore removes the hypothesis the proof uses, and Remark 5 offers no cover: that remark is a statement about λ and μ , whereas this is a perturbation of the gradient, which Remark 11 already identifies as the one perturbation that can raise $\hat{q}_s$.

A.3 The failure is systematic, not occasional

The substitution does not merely risk an invalid bound now and then. It drives the correction term to zero by construction, at every iterate, however unconverged.

Proposition 12 (The substituted correction collapses). *Let $\hat{v}$ satisfy the perturbed complementarity conditions of the barrier subproblem with parameter $\tau > 0$, that is $z_{L,i}(\hat{v}_i - l_i) = \tau$ for every finite l_i and $z_{U,i}(u_i - \hat{v}_i) = \tau$ for every finite u_i , with $z_{L,i} = 0$ when $l_i = -\infty$ and $z_{U,i} = 0$ when $u_i = +\infty$. Write $d_i = z_{L,i} - z_{U,i}$, and let $\hat{q}_s^{\text{rc}}(\hat{v})$ denote the right side of (4) with d_i in place of $\partial_i \varphi_s(\hat{v})$. Unlike $\hat{q}_s(\hat{v})$, it is not a bound. Then every term of its sum, evaluated by (5), lies in $(-\tau, 0]$, so the substituted correction lies in $(-n\tau, 0]$ and*

$$\hat{q}_s^{\text{rc}}(\hat{v}) \geq \varphi_s(\hat{v}) - n\tau.$$

Proof. Fix i and write $\sigma = \hat{v}_i - l_i > 0$ and $\theta = u_i - \hat{v}_i > 0$ in the two-sided case. Then $d_i = \tau/\sigma - \tau/\theta$, so $d_i > 0$ exactly when $\sigma < \theta$. In that case (5) selects $t = l_i$ and the term is

$$d_i(l_i - \hat{v}_i) = -\sigma\left(\frac{\tau}{\sigma} - \frac{\tau}{\theta}\right) = -\tau\left(1 - \frac{\sigma}{\theta}\right) \in (-\tau, 0),$$

using $0 < \sigma < \theta$. The case $d_i < 0$ is symmetric with σ and θ exchanged, and $d_i = 0$ gives the term 0. If $u_i = +\infty$ then $z_{U,i} = 0$ and $d_i = \tau/\sigma > 0$, so (5) selects $t = l_i$ and the term is exactly $-\tau$; the case $l_i = -\infty$ is symmetric. Summing over i gives the claim. $\square$

Remark 13 (What Pyomo reports as the reduced cost). Through the `.sol` file Ipopt returns z_L as the suffix `ipopt.zL.out` and $-z_U$ as the suffix `ipopt.zU.out`. Pyomo’s `get_reduced_costs` for Ipopt does not add them. For each variable it returns whichever of the two has the larger absolute value, so it reports $z_{L,i}$ or $-z_{U,i}$, never $z_{L,i} - z_{U,i}$. At a KKT point with strict complementarity at most one of the two is nonzero and the rules agree; at a truncated barrier iterate both are positive and they differ. Under the hypotheses of Proposition 12, $\sigma < \theta$ gives $z_{L,i} > z_{U,i}$, so Pyomo reports $d_i = \tau/\sigma$ and the term is exactly $-\tau$. Every term then lies in $[-\tau, 0]$, and the conclusion $\hat{q}_s^{\text{rc}}(\hat{v}) \geq \varphi_s(\hat{v}) - n\tau$ holds unchanged.

This was checked under Pyomo 6.10.1 installed from PyPI, with Ipopt 3.13.2 and the `contrib ipopt` interface, on $\min \sum_i (x_i - c_i)^2$ over $[0, 1]^4$ with $c = (-0.5, 0.2, 0.9, 1.7)$. The suffix `ipopt.zU.out` came back nonpositive. Stopped after two iterations, the second variable had gradient -0.042 , $z_L - z_U = -0.012$, and Pyomo’s reduced cost -0.494 . Run to convergence, all three agreed to within 10^{-8} .

Two consequences make reporting $\hat{q}_s^{\text{rc}}$ worse than merely unsound.

First, the quantity returned is governed by the solver’s barrier parameter and not by the inexactness of the solve. As $\tau \rightarrow 0$ it converges to $\varphi_s(\hat{v})$, which under complementarity and feasibility is the returned objective value — the inner bound ruled out under *Why the returned objective value is not a bound here* in Section 1. A scheme that reports it is not a certificate; it is the number the certificate was built to replace, shaved by $n\tau$. What is lost is precisely the property recorded under *Inexact solves* in Section 7: that the correction term measures its own inexactness and grows as $\hat{v}$ moves away from a KKT point.

Second, the substitution silently defeats the unbounded-variable guard. When $u_i = +\infty$ the reported $z_{U,i}$ is zero, so $d_i \geq 0$ and (5) never selects the infinite endpoint. The $-\infty$ that the honest gradient produces for an unconverged component with a missing bound — the case discussed under *Unbounded variables* in Section 7, which the implementation reports as “no bound” — can never arise, so a finite value would be returned in exactly the situation where no finite bound is available.

A.4 A valid variant, and why it is dominated

Validity can be restored by paying for the residual (8) explicitly. For any $t \in B$, $|r^\top(t - \hat{v})| \leq \|r\|_\infty \|t - \hat{v}\|_1 \leq \|r\|_\infty \|\text{diam}(B)\|_1$, so subtracting $\varepsilon_d \|\text{diam}(B)\|_1$ from $\hat{q}_s^{\text{rc}}$ restores a valid bound whenever $\|r\|_\infty \leq \varepsilon_d$. Two cautions attach: the box must have finite diameter, which Section 7 shows is not guaranteed, and ε_d must be the *unscaled* dual infeasibility, since the reported convergence measure is scaled by default.

Even so, the variant is dominated. Since $\nabla \varphi_s(\hat{v}) = d + r$ by (8) and the infimum in (4) separates by coordinate,

$$\begin{aligned} \sum_{i=1}^n \inf_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v})(t - \hat{v}_i) &\geq \sum_{i=1}^n \inf_{t \in [l_i, u_i]} d_i(t - \hat{v}_i) + \sum_{i=1}^n \inf_{t \in [l_i, u_i]} r_i(t - \hat{v}_i) \\ &\geq \sum_{i=1}^n \inf_{t \in [l_i, u_i]} d_i(t - \hat{v}_i) - \|r\|_\infty \|\text{diam}(B)\|_1. \end{aligned}$$

So whenever the variant is valid, $\hat{q}_s(\hat{v}) \geq \hat{q}_s^{\text{rc}}(\hat{v}) - \varepsilon_d \|\text{diam}(B)\|_1$: the gradient form is at least as tight, and it needs no estimate of $\|r\|_\infty$.

A.5 Dualizing the bounds instead

A tidier way to use the reported bound multipliers without forfeiting validity is to put them into the Lagrangian itself, adding a reduced-cost term to (2) and keeping the infimum over B . The bound this produces is valid, and the question becomes one of tightness only. It never gains.

Proposition 14 (Dualizing the bounds is dominated). *Let $z_L, z_U \in \mathbb{R}_+^n$, with $z_{L,i} = 0$ when $l_i = -\infty$ and $z_{U,i} = 0$ when $u_i = +\infty$, and define*

$$\tilde{\varphi}_s(v) = \varphi_s(v) + \sum_{i: l_i > -\infty} z_{L,i}(l_i - v_i) + \sum_{i: u_i < +\infty} z_{U,i}(v_i - u_i).$$

Let $\tilde{q}_s(\hat{v})$ be (4) with $\tilde{\varphi}_s$ in place of φ_s . Under the hypotheses of Theorem 3,

- (i) $\tilde{q}_s(\hat{v}) \leq L_s(W_s)$;
- (ii) $\tilde{q}_s(\hat{v}) \leq \hat{q}_s(\hat{v})$, coordinate by coordinate;
- (iii) equality holds in (ii) when $z_L = z_U = 0$, and also at a KKT point of (1) taken with its own bound multipliers, where both sides equal $L_s(W_s)$.

Proof. The added terms are affine, so $\tilde{\varphi}_s$ is convex and differentiable wherever φ_s is, with $\nabla \tilde{\varphi}_s(\hat{v}) = \nabla \varphi_s(\hat{v}) - z_L + z_U$. On B each added term is ≤ 0 , so $\tilde{\varphi}_s \leq \varphi_s$ there. Theorem 3 applied to $\tilde{\varphi}_s$ then gives $\tilde{q}_s(\hat{v}) \leq \inf_B \tilde{\varphi}_s \leq \inf_B \varphi_s = q_s(\lambda, \mu) \leq L_s(W_s)$, which is (i).

For (ii), an affine term coincides with its own tangent plane, so for every t

$$\tilde{\varphi}_s(\hat{v}) + \nabla \tilde{\varphi}_s(\hat{v})^\top(t - \hat{v}) = \varphi_s(\hat{v}) + \nabla \varphi_s(\hat{v})^\top(t - \hat{v}) + \sum_i [z_{L,i}(l_i - t_i) + z_{U,i}(t_i - u_i)],$$

the sum running over the finite bounds. Every bracket is ≤ 0 for $t \in B$. Both sides are separable across coordinates, so minimizing coordinate i over $[l_i, u_i]$ gives, in the extended reals, a term of $\tilde{q}_s$ no larger than the corresponding term of $\hat{q}_s$.

For (iii), the case $z = 0$ is immediate. At a KKT point with its bound multipliers, (7) gives $\nabla \tilde{\varphi}_s(\hat{v}) = 0$, so the correction in $\tilde{q}_s$ vanishes, and complementarity gives $z_{L,i}(l_i - \hat{v}_i) = z_{U,i}(\hat{v}_i - u_i) = 0$, so $\tilde{\varphi}_s(\hat{v}) = \varphi_s(\hat{v})$. Proposition 6 then gives $\tilde{q}_s(\hat{v}) = \hat{q}_s(\hat{v}) = L_s(W_s)$. $\square$

The general principle is the familiar one that dualizing a constraint which is also kept in the inner minimization can only lower the dual function, so for fixed (λ, μ) the best choice of bound multipliers is $z = 0$, which is (2) as it stands. The gradient of $\tilde{\varphi}_s$ at $\hat{v}$ is the residual r of (8), so $\tilde{q}_s$ is the substitution of the previous subsections made rigorous: it is valid with no estimate of $\|r\|_\infty$ and no finite-diameter requirement on B . Proposition 14 shows that the rigour costs tightness and returns nothing.

The loss is not merely formal. At a barrier iterate satisfying the perturbed complementarity of Proposition 12, each finite bound contributes $-\tau$ to $\tilde{\varphi}_s(\hat{v})$ before the correction is applied. When the sign of r_i selects the far endpoint, the bracket in the proof costs up to $z_i(u_i - l_i)$ on top of that. Worse, the unbounded-variable guard fires more often. Take l_i finite and $u_i = +\infty$. Near convergence $\partial_i \varphi_s(\hat{v}) \approx z_{L,i} > 0$, so (5) selects the finite endpoint and the term of $\hat{q}_s$ is finite. The coefficient in $\tilde{q}_s$ is instead $r_i = \partial_i \varphi_s(\hat{v}) - z_{L,i}$, a small residual of either sign, and whenever $r_i < 0$ the term is $-\infty$. For variables bounded on one side only, $\tilde{q}_s$ therefore reports “no bound” roughly as often as not, in situations where $\hat{q}_s$ is finite.

A.6 Where a saving would actually come from

The gradient evaluation that $\hat{q}_s^{\text{rc}}$ would skip is a real cost, and not a negligible one: rebuilding φ_s and walking it takes about 3 seconds at 10^5 constraint rows, growing linearly at roughly $30 \mu\text{s}$ per row, so a million-row scenario would spend on the order of 30 seconds per iteration inside the certificate. (A figure from one machine, quoted for its order of magnitude.) But it is the cost of differentiating φ_s , and it is addressable without touching the mathematics. The remedy is a faster *exact* gradient — forming $\nabla f_s + W_s + \nabla g_s^\top \lambda + \nabla h_s^\top \mu$ through compiled automatic differentiation rather than by walking a Python expression tree, or caching the expression skeleton across iterations with the multipliers held in mutable parameters — either of which leaves every hypothesis of Theorem 3 intact. Cheapening the gradient is safe; approximating it is not.

B Does a barrier solve in Gurobi give a bound?

Section 1 rules out reading a bound off Ipopt’s returned value, and the natural follow-up is whether a solver that runs its own primal–dual interior point method — Gurobi, say, on the convex relaxation — is in a different position. It is in a better one, but not a different one. The number it reports is correct to within its tolerances, and is not certified.

B.1 Why it is not strictly a bound

Gurobi’s barrier carries a dual iterate alongside the primal one, and terminates on the relative primal–dual gap, `BarConvTol`, together with primal and dual residual tests, `FeasibilityTol` and `OptimalityTol`. Weak duality, Lemma 2 again, makes the dual objective a lower bound only when the dual iterate is exactly feasible, and it is feasible only to within those tolerances and in floating point. The residual that Section A calls r in (8) is small here rather than absent, so the same objection applies, scaled down.

Crossover, on by default for linear programs, does not close this. It returns a basic solution, but “optimal” still means dual feasible to within `OptimalityTol` in floating point. For quadratically constrained models there is no crossover at all, so what is available is the barrier’s own tolerance-level answer. Nor is the mixed-integer case different in kind: `ObjBound` is assembled

from node relaxations solved to the same tolerances, and the arithmetic is not directed-rounding safe.

B.2 Why it is nevertheless better than Ipopt’s returned value

Two differences matter. First, a barrier solve that terminates reports a *gap*, not merely a value: the dual iterate is maintained throughout, and the stopping test is applied to the primal–dual gap directly. Ipopt reports the objective at the point it stopped at, with termination tested on a scaled KKT residual, which is how the Hilbert QP of Remark 1 comes back labelled **optimal** while being wrong in the third significant digit. Second, the tolerances are tighter and are applied unscaled. On a reasonable model the reported bound is wrong, if at all, at the level of those tolerances rather than at the level of the objective.

Remark 15 (Measured, on Gurobi 13.0.2). Defaults as reported by the library: `BarConvTol` = 10^{-8} , `FeasibilityTol` = `OptimalityTol` = 10^{-6} , `BarQCPConvTol` = 10^{-6} , `Crossover` automatic. Three measurements on one machine, each on a problem whose optimum is known analytically.

model	exact optimum	reported value	ObjBound
LP, barrier + crossover	13	13.0	13.0
LP, barrier, no crossover	13	13.000000000409	12.999999999850
SOCP (no crossover available)	$\sqrt{2}$	1.414213571461	1.414213548883

In both barrier rows the reported value lies *above* the optimum and `ObjBound` lies below it, by around $4\text{e-}10$ and $2\text{e-}8$ respectively — the tolerance level, not the objective level, and the opposite of the picture in Remark 1. The quadratically constrained model exposed no basis, confirming that no crossover was performed. Three instances establish nothing in general; they are quoted to place the magnitude.

B.3 Whether the bound survives the modelling layer

A bound the solver computed is of no use if the interface discards it, and for continuous models two of Pyomo’s three Gurobi interfaces do exactly that.

Remark 16 (Measured, on Pyomo 6.9.5.dev0). The LP of Remark 15 was solved through each interface with `Method` = 2 and `Crossover` = 0, and `results.problem.lower_bound` read off.

interface	lower_bound	which attribute
gurobi (LP file)	12.999999999850	ObjBound
gurobi_direct (legacy)	13.000000000409	ObjVal
gurobi_persistent (legacy)	13.000000000409	ObjVal

The mechanism is in `pyomo/solvers/plugins/solvers/gurobi_direct.py`: when a model has no binary or integer variables, the interface assigns `ObjVal` to *both* `upper_bound` and `lower_bound`, and reads `ObjBound` only in the mixed-integer branches. So for a continuous model the dual bound Gurobi computed is present on the model object and discarded before it reaches the results object, and the number carried forward as a lower bound sits *above* the

optimum — $4.1\text{e-}10$ above, where the discarded `ObjBound` sits $1.5\text{e-}10$ below. That is Section 1’s objection again, at tolerance magnitude. The mixed-integer path is unaffected, and the newer interfaces under `pyomo/contrib/solver/solvers/` assign `ObjBound` unconditionally, falling back to $\mp\infty$ when the attribute is missing, which is the safe direction.

This is not a hypothetical route into `mpi-sppy`. `SP0pt.solve_one` takes each subproblem’s `outer_bound` from `results.Problem[0].Lower_bound` for a minimisation, and `SP0pt.Ebound` — the same sum as Proposition 7 — adds those up. A Lagrangian spoke whose scenario subproblems are continuous and which is driven through the legacy persistent interface therefore aggregates primal objective values labelled as dual bounds. The error is at the level of Remark 16 rather than of Remark 1, and it is bounded by the solver’s tolerances rather than by nothing at all, which is why it has gone unnoticed; it is nonetheless the wrong quantity. The spoke documented in this note is not affected, because it never reads a bound off the solver.

B.4 Making it rigorous

Theorem 3 does not care which solver produced its inputs. It asks for convexity, a box, a point $\hat{v}$ and multipliers $\lambda \geq 0$, μ , and Gurobi supplies the last two for a continuous model as readily as Ipopt does. Specialising to a linear program recovers the bound already written down in Section A,

$$b^\top y + \sum_j \min_{x_j \in [l_j, u_j]} (c - A^\top y)_j x_j,$$

valid for *any* y , which is the safe bound of Neumaier and Shcherbina [4]. Their treatment adds directed rounding, so that the floating-point evaluation is itself rigorous — the gap Remark 11 leaves open here, and the reason their construction is used to certify results in exact mixed-integer programming.

The limitation carries over unchanged. A variable whose reduced cost has the sign that selects a missing bound contributes $-\infty$, so finite bounds are needed exactly where Section 7 says they are.

C A candidate defect in Pyomo’s legacy Gurobi interface

Remark 16 is worth separating from the rest of Appendix B, because unlike everything else in this note it is a claim about someone else’s code that we have not reported upstream. This appendix states it in the form an issue would take, and records what has and has not been established. It is written to be deleted once the question is settled one way or the other.

The observation. In `pyomo/solvers/plugins/solvers/gurobi_direct.py`, the `results` object is populated from the solved model. When the model has no binary and no integer variables, both `problem.upper_bound` and `problem.lower_bound` are assigned `ObjVal`. `ObjBound` is read only in the two mixed-integer branches that follow. Gurobi does populate `ObjBound` for a continuous model — Remark 15 reads it from one — so a number the solver computed, and the only one of the two that is a bound on the optimal value, is discarded at the interface.

Why it matters. For a minimisation the surviving number lies above the optimum rather than below it, so a consumer treating `lower_bound` as a dual bound is handed an inner bound. The discrepancy is at the level of the solver’s tolerances, $4.1\text{e-}10$ against $1.5\text{e-}10$ in Remark 16,

and it will pass unnoticed in most uses. It does not pass unnoticed in this one: `SP0pt.solve_one` takes each subproblem’s `outer_bound` from exactly this field, and `SP0pt.Ebound` sums those into the quantity Proposition 7 certifies. A bound assembled from values that are individually on the wrong side of their own subproblem optimum is not the quantity that proposition is about.

What the convention appears to be. The same shape appears in the sibling interfaces, so this is a convention rather than an oversight in one file. The legacy CPLEX interface assigns the objective value to both bounds under the same “no binary or integer variables” test, and the legacy Xpress interface assigns `lpobjval` to both on its continuous path. The convention is defensible on its own terms: for a continuous convex problem solved to optimality the primal and dual values coincide *in exact arithmetic*, so reporting one number twice records that. The objection is that it is exactly the identification this note spends Section 1 declining to make, and that for Gurobi, unlike possibly for the other two, a distinct and more conservative number was available and was dropped.

The suggested change. In the continuous branch, read `ObjBound` into `lower_bound` for a minimisation and into `upper_bound` for a maximisation, keeping `ObjVal` on the other side, and fall back to `ObjVal` when the attribute is unavailable. That makes the continuous branch agree with the mixed-integer branches directly below it, and with the newer interfaces under `pyomo/contrib/solver/solvers/`, which assign `ObjBound` unconditionally. The change is conservative in the sense that it can only move a reported lower bound down.

What has not been established. Two things. Whether CPLEX and Xpress expose an analogous continuous-model bound attribute, and so whether the same change applies to them, was not checked. And no claim is made that any released mpi-sppy result is wrong: the effect is bounded by the solver’s tolerances, which is a different statement from the effect being zero.

Remark 17 (Reproduced on a release). Remark 16 was measured against a development check-out, so the same LP was run again under Pyomo 6.9.4 installed from PyPI into a clean environment, with `gurobipy 13.0.3`, `Method = 2` and `Crossover = 0`.

interface	reported bound	which attribute
<code>gurobi_direct</code> (legacy)	13.000000000409	<code>ObjVal</code>
<code>gurobi_persistent</code> (legacy)	13.000000000409	<code>ObjVal</code>
<code>contrib gurobi_direct</code>	12.999999999850	<code>ObjBound</code>
<code>contrib gurobi_persistent</code>	12.999999999850	<code>ObjBound</code>

The behaviour is therefore not an artefact of a development branch. The released file carries the same continuous-model branch, and the two newer interfaces report the dual bound on the same model, from which the attribute is plainly available. The block is also of long standing: it was already in this form when it was last restructured in 2021.

References

- [1] Frank, M. and Wolfe, P.: An algorithm for quadratic programming. *Naval Research Logistics Quarterly* **3**(1–2), 95–110 (1956).

- [2] Jaggi, M.: Revisiting Frank–Wolfe: projection-free sparse convex optimization. *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 427–435 (2013).
- [3] Gade, D., Hackeibel, G., Ryan, S.M., Watson, J.-P., Wets, R.J.-B. and Woodruff, D.L.: Obtaining lower bounds from the progressive hedging algorithm for stochastic mixed-integer programs. *Mathematical Programming* **157**(1), 47–67 (2016).
- [4] Neumaier, A. and Shcherbina, O.: Safe bounds in linear and mixed-integer linear programming. *Mathematical Programming* **99**(2), 283–296 (2004).
- [5] Boland, N., Christiansen, J., Dandurand, B., Eberhard, A., Linderoth, J., Luedtke, J. and Oliveira, F.: Combining progressive hedging with a Frank–Wolfe method to compute Lagrangian dual bounds in stochastic mixed-integer programming. *SIAM Journal on Optimization* **28**(2), 1312–1336 (2018).